

AQI Predictor

REST API Documentation

FastAPI Backend — Complete Endpoint Reference

Base URL: `http://localhost:8000`

API Version: `1.0.0`

Swagger UI: `http://localhost:8000/docs`

ReDoc: `http://localhost:8000/redoc`

Author: Charu Mittal

Date: April 2026

Protocol: HTTP/1.1

Content-Type: `application/json`

Method	Path	Tag	Purpose
GET	/health	health	Liveness probe
GET	/ready	health	Readiness probe
GET	/predict	predict	24-hour AQI forecast
GET	/cities	cities	Supported cities list
GET	/model-info	model	Model metadata
GET	/metrics	model	Prometheus scrape
POST	/admin/sync	admin	Manual OpenAQ ingestion
POST	/admin/prune	admin	Manual data pruning

Contents

1	Overview	2
1.1	Introduction	2
1.2	Base URLs	2
1.3	Authentication	2
1.4	Response Conventions	2
1.5	HTTP Status Codes	2
2	Health & Readiness	3
3	Prediction Endpoint	4
3.1	Query Parameters	4
3.2	Example Request	4
3.3	Response — 200 OK	4
3.4	Response Field Definitions	4
3.5	Advisory Bands	6
3.6	Error Responses	6
3.7	Internal Execution Flow	6
4	Cities Endpoint	7
4.1	Response — 200 OK	7
5	Model Info Endpoint	8
5.1	Response — 200 OK	8
5.2	Field Definitions	8
6	Prometheus Metrics	10
6.1	Custom Metrics	10
7	Admin Endpoints	11
8	Pydantic Schema Reference	13
8.1	common_schema.py	13
8.2	predict_schema.py	13
9	OpenAPI YAML Specification	14
10	Quick Reference	19

1. Overview

1.1. Introduction

The **AQI Predictor REST API** is a FastAPI application that serves 24-hour Air Quality Index forecasts for five major Indian cities. It is backed by a trained **LightGBM** model served via an MLflow model server, a **SQLite** live data buffer (`live_aqi_buffer.db`), and an Apache Airflow pipeline that ingests real-time sensor data from **OpenAQ v3**.

1.2. Base URLs

Environment	URL
Local	<code>http://localhost:8000</code>
Docker (container)	<code>http://backend:8000</code>
Swagger UI	<code>http://localhost:8000/docs</code>
OpenAPI JSON	<code>http://localhost:8000/openapi.json</code>

1.3. Authentication

The current version does **not** require authentication. All endpoints are accessible within the Docker network. For public exposure, add an API Gateway or OAuth2 bearer token layer.

1.4. Response Conventions

- All responses are `application/json` unless otherwise stated.
- Timestamps are **ISO 8601 UTC**: `2026-04-27T10:00:00Z`.
- Pollutant values are **floats**: $\mu\text{g}/\text{m}^3$ for all except CO (mg/m^3).
- AQI values are **non-negative integers** (0–500).
- On error, the body is always: `{"detail": "<message>"}`.

1.5. HTTP Status Codes

Code	Status	When Used
200	OK	Request succeeded
400	Bad Request	<code>city</code> not in supported list
422	Unprocessable Entity	Missing/invalid query param (FastAPI validation)
500	Internal Server Error	MLflow inference threw an exception
503	Service Unavailable	Insufficient live data after two backfill attempts, or live fetch failure

2. Health & Readiness

GET `/health` **health**

Liveness probe. Always returns 200 OK if the server process is running.

Response — 200 OK

```
{ "status": "ok" }
```

Field	Type	Description
status	string	Always "ok"

GET `/ready` **health**

Readiness probe. Checks if the MLflow model server at port 5001 is reachable.

Responses — 200 OK

Listing 1: Model loaded

```
{ "status": "ready", "model_loaded": true }
```

Listing 2: Model not loaded

```
{ "status": "degraded", "model_loaded": false }
```

Field	Type	Description
status	string	"ready" or "degraded"
model_loaded	boolean	true if MLflow /ping on port 5001 returns HTTP 200

3. Prediction Endpoint

GET `/predict` **predict**

Core forecasting endpoint. Fetches live sensor data, builds the *LightGBM* feature vector, runs *MLflow* inference, and returns a 24-hour AQI forecast.

3.1. Query Parameters

Parameter	Type	Required	Description
city	string	Yes	Must match a value from <code>/cities</code> . Case-sensitive. Example: Chennai
location_id	integer	Yes	OpenAQ v3 location ID for the monitoring station. Example: 3135

3.2. Example Request

```
curl "http://localhost:8000/predict?city=Chennai&location_id=3135"
```

3.3. Response — 200 OK

```
{
  "city": "Chennai",
  "station_id": "openaq3135",
  "station_name": "Manali, Chennai CPCB",
  "location_id": 3135,
  "current_aqi": 87,
  "primary_pollutant": "PM2.5",
  "last_updated": "2026-04-27T10:00:00Z",
  "forecast_24h": [85, 86, 88, 90, 91, 89, 87, 85, 84, 83,
    82, 81, 80, 79, 78, 80, 82, 84, 86, 87,
    88, 87, 86, 85],
  "pollutants": {
    "PM2_5": 42.3,
    "PM10": 68.1,
    "NO2": 18.5,
    "SO2": 4.2,
    "CO": 0.9,
    "O3": 31.7
  },
  "weekly_trend": [92, 88, 85, 87, 90, 86, 87],
  "advisory": "Air quality is satisfactory. Sensitive people should stay alert."
}
```

3.4. Response Field Definitions

Field	Description
<code>city</code>	Echoes the <code>city</code> query parameter
<code>station_id</code>	Internal ID: <code>openaq{location_id}</code>
<code>station_name</code>	CPCB human-readable station name
<code>location_id</code>	Echoes the <code>location_id</code> query parameter
<code>current_aqi</code>	Current AQI from latest sensor readings
<code>primary_pollutant</code>	Highest sub-index pollutant; one of: PM2.5, PM10, NO2, SO2, CO, O3
<code>last_updated</code>	ISO 8601 UTC timestamp of the latest sensor row
<code>forecast_24h</code>	Exactly 24 predicted AQI integers, H+1 through H+24, all clamped ≥ 0
<code>pollutants.PM2_5</code>	PM2.5 in $\mu\text{g}/\text{m}^3$
<code>pollutants.PM10</code>	PM10 in $\mu\text{g}/\text{m}^3$
<code>pollutants.NO2</code>	NO2 in $\mu\text{g}/\text{m}^3$
<code>pollutants.SO2</code>	SO2 in $\mu\text{g}/\text{m}^3$
<code>pollutants.CO</code>	CO in mg/m^3
<code>pollutants.O3</code>	O3 in $\mu\text{g}/\text{m}^3$
<code>weekly_trend</code>	7 daily average AQI integers, oldest first; sourced from <code>trend_7d</code> field or last 7 forecast values if unavailable
<code>advisory</code>	CPCB-band health advisory string

The Pydantic schema field is `PM2_5` (underscore, JSON-safe). The `primary_pollutant` string value uses dot notation: `"PM2.5"`. These are different fields.

3.5. Advisory Bands

AQI Range	Category	Advisory Text
0–50	Good	Air quality is good. Outdoor activity is safe.
51–100	Satisfactory	Air quality is satisfactory. Sensitive people should stay alert.
101–200	Moderate	Moderate pollution. Limit prolonged outdoor exertion.
201–300	Poor	Poor air quality. Consider reducing outdoor activities.
301–400	Very Poor	Very poor air quality. Avoid outdoor exposure if possible.
401+	Severe	Severe air quality. Stay indoors and use protection.

3.6. Error Responses

HTTP Code	Condition and Example Detail
400	City not in <code>trained_cities.json</code> : "Chennai not supported"
422	Missing <code>city</code> or <code>location_id</code> : FastAPI validation error JSON
503	After two backfill attempts, rows < 168: "Not enough live rows for Chennai 3135: 45 < 168"
503	OpenAQ live fetch threw exception: "Live fetch failed for Chennai 3135: ..."
500	MLflow inference exception: "Inference failed: ..."

3.7. Internal Execution Flow

1. Validate `city` against `trained_cities.json` → **400** if absent.
2. Try to fetch the latest OpenAQ timestamp for `location_id` (non-fatal if this fails).
3. Count SQLite `city_readings` rows for this station over the last **200 hours**.
4. If rows < 168 **or** DB row is stale by >1 hour → trigger **incremental backfill**.
5. Re-count rows. If still < 168 → trigger **force-refresh backfill**.
6. Re-count rows. If still < 168 → raise **503**.
7. Call `build_live_payload(city, hours=200, location_id)` → feature matrix **X**.
8. POST **X** to MLflow server at `:5001/invocations` → 24 raw floats.
9. Clamp negatives to 0, round to int, pad/trim to exactly 24 values.
10. Persist result to `predictions` table in SQLite.
11. Update Prometheus counters (`PREDICT_REQUESTS`, `PREDICT_LATENCY`, `LATEST_DB_AGE_HOURS`) in the `finally` block.
12. Return `PredictResponse`.

4. Cities Endpoint

GET `/cities` **cities**

Returns the list of cities the *LightGBM* model was trained on.

4.1. Response — 200 OK

```
{
  "cities": ["Chennai", "Delhi", "Mumbai", "Kolkata", "Ahmedabad"]
}
```

Field	Type	Description
cities	string[]	Loaded at startup from <code>deployment_bundle/preprocessing/trained_cities.json</code>

The five supported cities are: **Chennai, Delhi, Mumbai, Kolkata, Ahmedabad**.
The Streamlit UI calls this endpoint on page load to populate the city dropdown.

5. Model Info Endpoint

GET /model-info **model**

Returns metadata about the currently loaded model: version, MLflow run, metrics, and forecast configuration.

5.1. Response — 200 OK

```
{
  "app_name": "AQI_Forecast_API",
  "app_version": "1.0.0",
  "model_name": "AQIModel",
  "model_version": "3",
  "model_stage": "Production",
  "mlflow_run_id": "abc123def456...",
  "model_source": "mlflow",
  "model_family": "lightgbm",
  "git_tag": "v1.0.0",
  "git_commit": "def456abc...",
  "primary_metric": "rmse",
  "primary_metric_value": 12.4,
  "forecast_horizon_hours": 24,
  "supported_cities_count": 5,
  "supported_cities": ["Chennai", "Delhi", "Mumbai",
    "Kolkata", "Ahmedabad"]
}
```

5.2. Field Definitions

Field Source / Description	Type
app_name config.APP_NAME = "AQI Forecast API"	string
app_version config.APP_VERSION = "1.0.0"	string
model_name metadata["model_name"] or config.MODEL_NAME	string
model_version metadata["model_version"]	string
model_stage metadata["model_stage"] (e.g. "Production")	string
mlflow_run_id metadata["mlflow_run_id"]	string
model_source metadata["model_source"]	string
model_family metadata["model_family"] — always "lightgbm" (or "unknown" if missing)	string

Field Source / Description	Type
git_tag	string
metadata["git_tag"] — Git tag at training time	
git_commit	string
metadata["git_commit"] — SHA at training time	
primary_metric	string
metadata["primary_metric"]	
primary_metric_value	float
metadata["primary_metric_value"]	
forecast_horizon_hours	integer
metadata["forecast_horizon"] or default 24	
supported_cities_count	integer
len(get_supported_cities())	
supported_cities	string[]
get_supported_cities() from trained_cities.json	

6. Prometheus Metrics

GET `/metrics` **model**

Prometheus scrape endpoint. Returns custom application metrics in plain text exposition format.

Response Content-Type

text/plain; version=0.0.4; charset=utf-8

6.1. Custom Metrics

Metric Description	Type	Labels
<code>aqi_predict_requests_total</code> Total /predict calls labelled ok/error	Counter	city, status
<code>aqi_predict_latency_seconds</code> End-to-end /predict latency. Buckets: 0.1, 0.25, 0.5, 1, 2, 5, 10, 30s	Histogram	city
<code>aqi_live_fetch_requests_total</code> OpenAQ live fetch calls	Counter	city, status
<code>aqi_live_fetch_rows_total</code> SQLite rows written per live fetch	Counter	city
<code>aqi_latest_db_age_hours</code> Age of the most recent <code>city_readings</code> row in hours	Gauge	city
<code>aqi_drift_psi</code> PSI per feature from latest <code>drift_results</code> run	Gauge*	feature
<code>aqi_drift_detected</code> 1.0 if drift flagged; 0.0 otherwise	Gauge*	feature
<code>aqi_drift_features_count</code> Total features currently drifted	Gauge*	—
<code>aqi_model_ready</code> 1 = MLflow server reachable; 0 = not ready	Gauge	—

Metrics marked * are served by a custom `DriftCollector` class that reads from the `drift_results` SQLite table **on every Prometheus scrape**. This is a pull-based design: Airflow writes drift results to SQLite; FastAPI reads and serves them on demand. No cross-process Prometheus push is needed.

7. Admin Endpoints

Admin endpoints trigger expensive operations. Use only for manual maintenance or debugging. They are **not** called by the Streamlit frontend.

POST `/admin/sync` **admin**

Manually triggers a full OpenAQ sensor ingestion cycle via `sync_all_sensors()`.

Request

No query parameters. No request body.

Response — 200 OK

```
{
  "results": { ... }
}
```

`sync_all_sensors()` in `ingestion_service.py` returns a dict summarising the run (sensor counts, rows written, errors per station). The exact shape depends on your ingestion service implementation. The router wraps it as: `return {"results": sync_all_sensors()}`.

POST `/admin/prune` **admin**

Deletes `hourly_measurements` rows older than `retention_days` via `prune_old_measurements()`.

Query Parameters

Parameter	Type	Required	Description
<code>retention_days</code>	integer	No (default: 90)	Delete rows older than this many days

Response — 200 OK

```
{
  "deleted": true,
  "retention_days": 90
}
```

Field	Type	Description
<code>deleted</code>	boolean	<code>true</code> when pruning completes successfully
<code>retention_days</code>	integer	The retention window that was applied

`prune_old_measurements(retention_days)` calls `cleanup_old_hourly_measurements(days=retention_days)` and returns `{"pruned": True, "retention_days": retention_days}`. The admin router exposes this directly.

8. Pydantic Schema Reference

All schemas live in `backend/app/schemas/`.

8.1. `common_schema.py`

Listing 3: HealthResponse

```
{ "status": "ok" }
```

Listing 4: ReadyResponse

```
{ "status": "ready", "model_loaded": true }
```

Listing 5: CitiesResponse

```
{ "cities": ["Chennai", "Delhi", "Mumbai", "Kolkata", "Ahmedabad"] }
```

8.2. `predict_schema.py`

Listing 6: PollutantsResponse

```
{  
  "PM2_5": 42.3, "PM10": 68.1, "NO2": 18.5,  
  "SO2": 4.2, "CO": 0.9, "O3": 31.7  
}
```

Listing 7: PredictResponse (all fields)

```
{  
  "city": "string",  
  "station_id": "string|_null",  
  "station_name": "string|_null",  
  "location_id": "integer|_null",  
  "current_aqi": "integer",  
  "primary_pollutant": "string",  
  "last_updated": "string",  
  "forecast_24h": "[integer_x24]",  
  "pollutants": "PollutantsResponse",  
  "weekly_trend": "[integer]",  
  "advisory": "string"  
}
```

9. OpenAPI YAML Specification

Save as `openapi.yaml` in the project root.

```
openapi: "3.1.0"
info:
  title: "AQI_Forecast_API"
  version: "1.0.0"
  description: >
    FastAPI backend for 24-hour AQI forecasting for 5 Indian cities.
    Backed by LightGBM served via MLflow, SQLite live buffer,
    and OpenAQ v3 ingestion via Apache Airflow.
  contact:
    name: "Charu_Mittal"
    email: "mittalcharu.59@gmail.com"

servers:
- url: "http://localhost:8000"
  description: "Local_development"
- url: "http://backend:8000"
  description: "Docker_(container-to-container)"

tags:
- name: health
- name: predict
- name: cities
- name: model
- name: admin

paths:
  /health:
    get:
      tags: [health]
      summary: "Liveness_probe"
      responses:
        "200":
          content:
            application/json:
              schema: { $ref: "#/components/schemas/HealthResponse" }
              example: { status: "ok" }

  /ready:
    get:
      tags: [health]
      summary: "Readiness_probe"
      responses:
        "200":
          content:
            application/json:
              schema: { $ref: "#/components/schemas/ReadyResponse" }

  /predict:
    get:
      tags: [predict]
      summary: "24-hour_AQI_forecast"
      parameters:
        - name: city
```

```

    in: query
    required: true
    schema: { type: string }
    example: "Chennai"
  - name: location_id
    in: query
    required: true
    schema: { type: integer }
    example: 3135
responses:
  "200":
    content:
      application/json:
        schema: { $ref: "#/components/schemas/PredictResponse" }
  "400":
    content:
      application/json:
        schema: { $ref: "#/components/schemas/ErrorResponse" }
        example: { detail: "Chennai_not_supported" }
  "422":
    content:
      application/json:
        schema: { $ref: "#/components/schemas/ValidationError" }
  "503":
    content:
      application/json:
        schema: { $ref: "#/components/schemas/ErrorResponse" }
        example:
          detail: "Not_enough_live_rows_for_Chennai_3135:_45_168"
  "500":
    content:
      application/json:
        schema: { $ref: "#/components/schemas/ErrorResponse" }

/cities:
  get:
    tags: [cities]
    summary: "List_supported_cities"
    responses:
      "200":
        content:
          application/json:
            schema: { $ref: "#/components/schemas/CitiesResponse" }
            example:
              cities: [Chennai, Delhi, Mumbai, Kolkata, Ahmedabad]

/model-info:
  get:
    tags: [model]
    summary: "Model_metadata"
    responses:
      "200":
        content:
          application/json:
            schema: { $ref: "#/components/schemas/ModelInfoResponse" }

/metrics:
  get:

```



```
tags: [model]
summary: "Prometheus_metrics_scrape"
responses:
  "200":
    content:
      text/plain:
        schema: { type: string }

/admin/sync:
  post:
    tags: [admin]
    summary: "Trigger_manual_OpenAQ_ingestion"
    responses:
      "200":
        content:
          application/json:
            schema:
              type: object
              properties:
                results: { type: object }

/admin/prune:
  post:
    tags: [admin]
    summary: "Prune_old_hourly_measurements"
    parameters:
      - name: retention_days
        in: query
        required: false
        schema: { type: integer, default: 90 }
    responses:
      "200":
        content:
          application/json:
            schema:
              type: object
              properties:
                deleted: { type: boolean }
                retention_days: { type: integer }
            example:
              deleted: true
              retention_days: 90

components:
  schemas:

    HealthResponse:
      type: object
      properties:
        status: { type: string, enum: [ok] }

    ReadyResponse:
      type: object
      properties:
        status: { type: string, enum: [ready, degraded] }
        model_loaded: { type: boolean }

    CitiesResponse:
```

```
type: object
properties:
  cities:
    type: array
    items: { type: string }

PollutantsResponse:
  type: object
  properties:
    PM2_5: { type: number }
    PM10: { type: number }
    NO2: { type: number }
    SO2: { type: number }
    CO: { type: number }
    O3: { type: number }

PredictResponse:
  type: object
  properties:
    city: { type: string }
    station_id: { type: string, nullable: true }
    station_name: { type: string, nullable: true }
    location_id: { type: integer, nullable: true }
    current_aqi: { type: integer }
    primary_pollutant: { type: string }
    last_updated: { type: string, format: date-time }
    forecast_24h:
      type: array
      items: { type: integer }
      minItems: 24
      maxItems: 24
    pollutants: { $ref: "#/components/schemas/PollutantsResponse" }
    weekly_trend:
      type: array
      items: { type: integer }
    advisory: { type: string }

ModelInfoResponse:
  type: object
  properties:
    app_name: { type: string }
    app_version: { type: string }
    model_name: { type: string }
    model_version: { type: string }
    model_stage: { type: string }
    mlflow_run_id: { type: string }
    model_source: { type: string }
    model_family: { type: string }
    git_tag: { type: string }
    git_commit: { type: string }
    primary_metric: { type: string }
    primary_metric_value: { type: number }
    forecast_horizon_hours: { type: integer }
    supported_cities_count: { type: integer }
    supported_cities:
      type: array
      items: { type: string }
```

```
ErrorResponse:
  type: object
  properties:
    detail: { type: string }

ValidationError:
  type: object
  properties:
    detail:
      type: array
      items:
        type: object
        properties:
          loc: { type: array, items: { type: string } }
          msg: { type: string }
          type: { type: string }
```

10. Quick Reference

Method	Path	Codes	Purpose
GET	/health	200	Is the process alive?
GET	/ready	200	Is the MLflow server reachable?
GET	/predict?city=&location_id=	200, 400, 422, 500, 503	24h AQI forecast
GET	/cities	200	List of 5 supported cities
GET	/model-info	200	Model version and metrics
GET	/metrics	200	Prometheus scrape
POST	/admin/sync	200	Run ingestion manually
POST	/admin/prune?retention_days=	200	Prune old sensor rows

Listing 8: Quick curl test

```
curl http://localhost:8000/health
curl http://localhost:8000/ready
curl "http://localhost:8000/predict?city=Chennai&location_id=3135"
curl http://localhost:8000/cities
curl http://localhost:8000/model-info
curl -X POST http://localhost:8000/admin/sync
curl -X POST "http://localhost:8000/admin/prune?retention_days=30"
```